HiveMind: Autonomous Logistics Simulator

Academic Context:
This project was developed for the Object-Oriented Programming (OOP) course at the Politehnica University of Bucharest, Faculty of Automatica si Calculatoare.

---

1. Project Overview

It's the year 2030. Urban logistics is managed by autonomous fleets. You are the Software Architect for the "HiveMind" system, a simulation and decision-making engine that coordinates a heterogeneous fleet of robots (Drones, Ground Robots, Scooters).

Main Objective:
Create a modular C++ application that:

1. Procedurally generates city maps (environment simulations).
2. Simulates agent physics and lifecycle.
3. Optimizes profit through a custom task allocation algorithm (HiveMind), demonstrating "out of the box" algorithmic thinking.

---

2. Project Structure

```
.
├── compile.txt                 // Compilation instructions
├── documentation               // Project documentation
│   ├── demo.mp4                // 3-minute functionality demo
│   ├── diagrama.png            // Class Diagram
│   └── documentatie.pdf        // Detailed technical report
├── main.cpp                    // Entry point
├── maps                        // Storage for map files
├── output                      // Generated results
│   ├── debug_map.txt           // Last generated/loaded map
│   └── strategy_comparison.txt // Performance metrics for all algorithms
├── README.md                   // This file
├── simulation_setup.txt        // Global configuration parameters
└── src
```

```
├── agents                  // Agent hierarchy & Factory
├── core                    // Simulation internals
├── logic                   // Business logic
├── map                     // The "Genesis" Module
├── pathfinding             // Navigation algorithms
└── strategies              // The "HiveMind" Module
```

---

3. The "Genesis" Module: Environment Generation

The world is a grid of size **N * M**. We utilize the **Strategy Design Pattern** to support various map generation methods:

FileMapLoader

- Loads maps from text files, following a specific format.

Procedural Generators

- **Random:** Randomly places obstacles based on a density parameter.
- **Archipelago:** Creates clusters of obstacles (islands) based on a density parameter.
- **Maze:** Implements Recursive Backtracking for structured corridors.
- **District:** Generates urban blocks separated by straight streets.
- **Radial:** Creates a central plaza (Hub) with concentric ring roads.
- **Canyon:** Big, open corridor in the center with walls on sides.

Battery-Aware Validation

Validation considers agent physics, not just connectivity:

- **Standard Validation:** Multi-stage BFS calculates maximum range of a ground unit (Robot). Charging Stations act as "relay points" to extend range.
- **Drone Validation:** Similarly to the Standard, but Drones ignore walls.

---

4. The "HiveMind" Logic

All the strategies implement the **IDispatchStrategy** interface, allowing easy swapping and comparison. They are responsible for assigning packages to agents to maximize profit.

- **Greedy:** Basic "closest agent, available package" logic.

- **Greedy Optimized:** Adds speed-based priority (Drones first) and a simple range check.
- **Charging:** Routes agents to intermediate Charging Stations if battery is low.
- **No Deaths:** Similar to **Greedy Optimized**, but caches A* paths and calculates precise battery needs, eliminating penalties for dead agents.

---

5. Technical Implementation Details

Pathfinding

Each agent type has a dedicated pathfinding algorithm, implemented via the **Strategy Design Pattern**:

- **A* Algorithm:** Ground agents (Robots/Scooters) using Manhattan distance heuristic.
- **Linear Algorithm:** Drones ignore walls, moving straight/diagonal to target.

Design Patterns

- **Singleton:** `ConfigManager` for global parameters.
- **Factory:** `AgentFactory` creates Drones, Robots, Scooters.
- **Strategy:** Applied to `IMapGenerator`, `IDispatchStrategy`, `IPathAlgorithm`.

Performance & Fairness

- **Deterministic Seed:** `PackageManager` admits a seed value for reproducible results.

Error Handling

- Custom exception hierarchy manages simulation-critical errors (missing files, malformed maps, failed generation).

---

6. Simulation Output

- **Live Rendering:** Terminal interface with agent movement, battery levels, paths, package status and financial telemetry.
- **Strategy Comparison:** At the end, all strategies run in headless mode and results are saved in `output/strategy_comparison.txt`.

Profit Formula:

```
Profit = Σ(Rewards) - Σ(Operating Costs) - Σ(Penalties)
```

Where:

- **Rewards:** Based on package priority and distance delivered.
- **Operating Costs:** Proportional to distance traveled and agent type.
- **Penalties:** Incurred for undelivered packages and dead agents:
  - Dead Agent: -500
  - Undelivered Package: -200
  - Late Delivery: -50

---

7. How to Run

Compilation

Ensure a C++20-compatible compiler is used:

```
g++ -std=c++20 -o HiveMind.bin src/*/*.cpp main.cpp
```

Execution

1. Configure parameters in `simulation_setup.txt`.
2. Run the simulation:

```
./HiveMind.bin
```